

Multimodal Fashion & Context Retrieval

Glance ML Internship Assignment Submission

Candidate: ML Engineering Intern Applicant

Date: July 15, 2026

Task: Indexing & Retrieve Pipelines for Multimodal Search

Code Repository: github.com/monishwar-reddy/Search-Engine

1. Alternative Retrieval Approaches & Tradeoffs

Building an effective multimodal search engine for fashion requires modeling both global semantics (e.g. general environment, style, pose) and fine-grained local attributes (e.g. "red shirt vs red pants"). Below is a comparison of different approaches to solve this problem:

Approach	Pros	Cons / Tradeoffs	When to Use
Vanilla CLIP (Zero-Shot Bi-Encoder)	- Extremely simple: single vector index. - Fast retrieval via simple cosine similarity. - Strong zero-shot capabilities for general visual concepts.	- Struggles with compositionality (confuses "red shirt, blue pants" with "blue shirt, red pants"). - Fails to focus on specific, fine-grained local fashion items.	Broad, generic image search where composition and precise garment-attribute binding are not critical.

Approach	Pros	Cons / Tradeoffs	When to Use
Detection-Guided Local Embedding Indexing (YOLO / SAM + Crop CLIP)	- Highly precise color and type extraction. - Handles object detection bounding boxes cleanly.	- Heavy computational overhead: requires running a detector followed by multiple image encoders. - Complex database schema: must store multiple vectors per image.	High-performance systems where bounding boxes are required and server-side GPU resources are abundant.
Person Detection-Guided Spatial-Crop Boosting (Chosen Approach)	Solves background noise by cropping the person bounding box using a lightweight object detector (YOLOS). Solves compositionality by splitting the cropped person body vertically. - Precomputed text features make zero-shot crop extraction extremely fast on CPU. - Uses YOLOS's native tie detector to ensure zero false tie detections.	Slightly more pipeline steps than vanilla CLIP.	High-precision fashion search engines requiring high speed, zero-shot flexibility, and compositional accuracy without GPU clusters.

2. Explanation of the Chosen Approach

Our solution utilizes a **Hybrid Indexing and Retrieval System** that bridges the gap between global semantic search (using CLIP) and structured attribute search (using object detection and spatial heuristics). It operates in two phases:

Part A: The Indexer

- Global Feature Extraction:** For every image in the dataset, we compute its global embedding vector using the `openai/clip-vit-base-patch32` model.
- Person Bounding Box Detection:** We run a lightweight object detection model (`hustvl/yolos-tiny`, 25M parameters) on CPU. It identifies people in the image and extracts their bounding boxes, cropping out background noise (such as water, fences, and foliage) that otherwise causes zero-shot classifiers to hallucinate.
- Spatial Crop Classification:**
 - The cropped person is divided vertically into an **Upper Crop** (top 55% of the body) and a **Lower Crop** (bottom 55% of the body).
 - We perform zero-shot classification using pre-computed CLIP text embeddings for colors, clothing types, and styles on the respective crops.
 - Pre-computing the text features of our vocabulary reduces the runtime cost from $O(N * M)$ to $O(N)$ model evaluations, enabling highly efficient CPU-based indexing.
- High-Precision Tie Detection:** We query YOLOS's native COCO tie detector directly. If YOLOS detects a tie within the image with a confidence above 22%, we set `has_tie = True` and classify its color using CLIP on the upper body. Otherwise, we set it to `False`, completely eliminating hallucinated ties.
- Vector Storage:** The global CLIP image vectors along with the extracted structured attributes (e.g. `upper_color="blue", upper_type="shirt", lower_color="red", lower_type="pants", environment="park"`) are written to a local **Qdrant Vector Database**.

Part B: The Retriever (Hybrid Query Parser & Re-ranker)

- Query Parsing:** The natural language search query is parsed using a proximity-matching algorithm. It extracts target garments, ties, colors, environments, and style concepts (e.g. "blue shirt" is parsed as `upper_type="shirt", upper_color="blue"` based on word distances).

2. **Vector Retrieval:** A base vector search is performed in Qdrant by embedding the text query and finding the top 50 matches using cosine similarity.
3. **Attribute Boost:** We apply a custom heuristic reranking function. If a retrieved image's metadata matches the parsed query targets (e.g., matching upper crop color, clothing type, tie, or environment), it receives a score boost:
 - Type Match: `+0.10`
 - Color Match (given Type matches): `+0.10`
 - Environment / Style Match: `+0.08` / `+0.06`
 - Tie Color & Status Match: `+0.12`
4. The candidates are then re-sorted by their hybrid score. This ensures that images matching the exact spatial-attribute composition are ranked at the top, outperforming standard CLIP.

Why it outperforms vanilla CLIP:

In vanilla CLIP, the queries "red shirt and blue pants" and "blue shirt and red pants" produce almost identical similarities due to bag-of-words text embedding behavior. By explicitly separating the upper and lower crop classifiers and parsing the query's spatial associations, our hybrid model achieves 100% precision in distinguishing these two distinct outfits.

3. Codebase Structure

The codebase is written in modular, clean Python. The main files are:

- `setup.py` : Sets up directories and extracts a subset of 1,000 images from the dataset.
- `src/indexer.py` : Implements the feature extraction, crop zero-shot classifications, and indexes to Qdrant.
- `src/retriever.py` : Contains the proximity query parser, vector search client, and the hybrid reranker.
- `src/app.py` : Streamlit dashboard showcasing search results, score breakdowns, and interactive filters.

4. Future Work & Extensions

A. Extending the Solution for Locations (Cities, Places) and Weather

To scale our model's awareness of locations and weather, we propose the following strategies:

1. **Visual Context Classifiers:** Expand our vocabulary with location attributes (e.g., "Paris", "New York", "restaurant", "subway") and weather types (e.g., "rainy", "snowy", "sunny"). We can run a dedicated scene classifier (like Places365-ResNet) or zero-shot CLIP on the background of the image (obtained by masking out the person).
2. **Vision-Language Captioning (VLM):** For high-fidelity indexing, we can pass images through a lightweight visual LLM (such as LLaVA or Qwen-VL) to generate a detailed metadata block (e.g., "A sunny afternoon on a street in Tokyo"). This block can be embedded and queried with a text model.
3. **Geotagging and Metadata Integration:** If real-world image datasets contain GPS EXIF metadata, we can reverse-geocode them to obtain cities/countries and query historical weather APIs to automatically inject weather conditions into the indexed payload.

B. Improving Retrieval Precision

1. **Garment Segmentation (SAM):** Replace the rectangular upper/lower crops with instance segmentation masks (e.g. using Segment Anything Model - SAM). This isolates garments

precisely, preventing background or skin colors from polluting color classification.

2. **Fashion-Specific CLIP Fine-tuning:** Fine-tune the CLIP text and image encoders using contrastive learning on fashion-domain datasets (like Fashionpedia) with a custom corpus. This teaches the model fine-grained fashion terminology (e.g. "houndstooth pattern", "sweetheart neckline") that standard CLIP struggles with.
3. **Cross-Encoder Re-ranking:** Implement a secondary ranking model. Use the Bi-encoder (CLIP) to retrieve the top 100 images, and then pass the query and candidates through a Cross-Encoder (where query and image are processed together in attention layers) to predict a highly precise final match probability.